

Cosa imparerai oggi

In questa lezione esploreremo uno degli algoritmi fondamentali del Machine Learning: la regressione logistica. Partiremo dalle basi per costruire una comprensione solida e pratica di questo potente strumento di classificazione.

01

Regressione vs Classificazione

Capire la differenza fondamentale tra problemi di regressione e classificazione, e quando usare ciascun approccio

02

Limiti della Regressione Lineare

Perché la regressione lineare non funziona per i problemi di classificazione e quali problemi genera

03

La Funzione Sigmoidale

Come funziona la funzione sigmoide e perché è perfetta per trasformare valori in probabilità

04

Costruzione del Modello

Come si costruisce il modello di regressione logistica combinando funzioni lineari e sigmoide

05

Decision Boundary

Come interpretare il confine decisionale, sia nei casi lineari che non lineari

Che cos'è la Classificazione?

La classificazione è un tipo di problema di Machine Learning in cui l'obiettivo è predire a quale categoria appartiene un'osservazione. A differenza della regressione, dove l'output è un valore continuo (come un prezzo o una temperatura), nella classificazione l'output y può assumere solo **valori discreti** che rappresentano categorie o classi specifiche.



Filtro Spam

Questa email è spam?

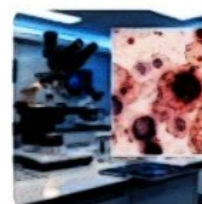
- Classe 0: Email legittima
- Classe 1: Spam



Frodi Finanziarie

La transazione è fraudolenta?

- Classe 0: Transazione legittima
- Classe 1: Frode



Diagnosi Medica

Il tumore è maligno?

- Classe 0: Benigno
- Classe 1: Maligno

☐ **ATTENZIONE:** I termini "negativo" ($y=0$) e "positivo" ($y=1$) sono convenzioni matematiche e NON significano "cattivo" e "buono"! Rappresentano semplicemente l'assenza o la presenza della caratteristica che stiamo cercando di identificare.

Il Dataset di Esempio: Tumori

Training set

	tumor size (cm)	...	patient's age	malignant?	$i = 1, \dots, m$ training examples $j = 1, \dots, n$ features
	x_j		x_i	y	
1st	10		52	1	target y is 0 or 1
...	2		73	0	
...	5		55	0	
last	12		49	1	
	...		...	...	

Per comprendere concretamente come funziona la classificazione, utilizzeremo un dataset medico reale che contiene informazioni su pazienti con tumori. Questo è un classico esempio di **Training Set** per la classificazione binaria, dove l'obiettivo è imparare a distinguere tra tumori benigni e maligni basandosi su caratteristiche misurabili.

Features (x)

Le variabili di input che usiamo per fare previsioni:

- **Dimensione del tumore:** misurata in centimetri o millimetri
- **Età del paziente:** un fattore di rischio importante
- Altre caratteristiche cliniche rilevanti

Target (y)

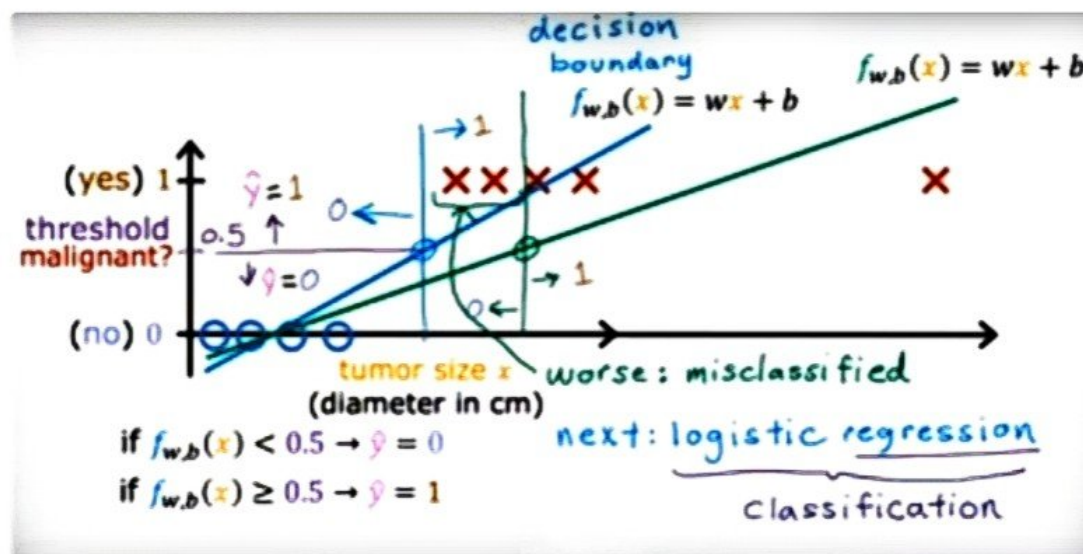
La variabile che vogliamo predire rappresenta la diagnosi:

- **y = 0:** Tumore benigno (non canceroso)
- **y = 1:** Tumore maligno (canceroso)

Il modello imparerà dai dati storici etichettati per fare previsioni accurate su nuovi pazienti.

Perché la Regressione Lineare NON Funziona

Quando affrontiamo un problema di classificazione, potremmo essere tentati di usare una semplice regressione lineare con la formula $f(x) = wx + b$. Tuttavia, questo approccio presenta problemi fondamentali che lo rendono inadatto per la classificazione.



⚠ Apparente Funzionalità

Inizialmente, fissando una soglia a 0.5 per separare le classi, il modello lineare sembra funzionare ragionevolmente bene sui dati di training.

✗ Problema degli Outlier

Basta un solo dato anomalo o estremo per spostare significativamente la retta di regressione, causando errori di classificazione su punti che prima erano correttamente classificati.

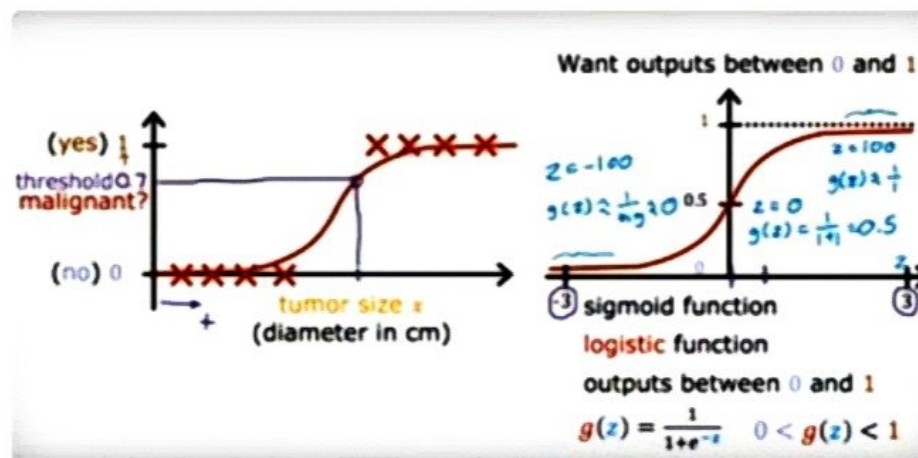
✗ Valori Impossibili

La retta può predire valori maggiori di 1 o minori di 0, che non hanno alcun senso quando interpretiamo l'output come una probabilità (le probabilità devono essere comprese tra 0 e 1).

Conclusione: Serve un modello diverso che produca sempre output compresi tra 0 e 1 e che sia più robusto agli outlier. La soluzione è la [Regressione Logistica](#).

La Funzione Sigmoidale (Logistic Function)

La funzione sigmoide, anche chiamata funzione logistica, è il componente matematico chiave che rende possibile la regressione logistica. Questa elegante funzione trasforma qualsiasi valore reale in un output compreso tra 0 e 1, perfetto per rappresentare probabilità.



La Formula

$$g(z) = \frac{1}{1 + e^{-z}}$$

Dove z può essere qualsiasi numero reale (da $-\infty$ a $+\infty$), ed e è il numero di Nepero (≈ 2.718).

Proprietà Fondamentali

- **Output limitato:** Il risultato è sempre compreso tra 0 e 1, perfetto per le probabilità
- **Comportamento asintotico:** Quando $z \rightarrow +\infty$, $g(z) \rightarrow 1$; quando $z \rightarrow -\infty$, $g(z) \rightarrow 0$
- **Punto centrale:** Quando $z = 0$, $g(z) = 0.5$ esattamente
- **Forma a "S":** La curva ha una caratteristica forma sigmoide che transita dolcemente da 0 a 1

Costruire il Modello

La regressione logistica combina l'approccio lineare che già conosciamo con la funzione sigmoide, creando un modello potente ma interpretabile. Il processo avviene in due fasi sequenziali che trasformano le features di input in una probabilità di output.

1

Passo 1: Combinazione Lineare

Calcolare la combinazione lineare delle features, esattamente come nella regressione lineare:

$$z = w \cdot x + b$$

Dove **w** sono i pesi, **x** le features e **b** il bias.

2

Passo 2: Applicare la Sigmoid

Applicare la funzione sigmoide al risultato per ottenere una probabilità:

$$f(x) = g(z) = \frac{1}{1 + e^{-z}}$$

Questo trasforma z in un valore tra 0 e 1.

I

Interpretation of logistic regression output

$$f_{\bar{w},b}(\bar{x}) = \frac{1}{1 + e^{-(\bar{w} \cdot \bar{x} + b)}}$$

"probability" that class is 1

Example:

$\bar{x}$ is "tumor size"
 y is 0 (not malignant)
or 1 (malignant)

$f_{\bar{w},b}(\bar{x}) = 0.7$
70% chance that y is 1

$$f_{\bar{w},b}(\bar{x}) = P(y = 1 | \bar{x}; \bar{w}, b)$$

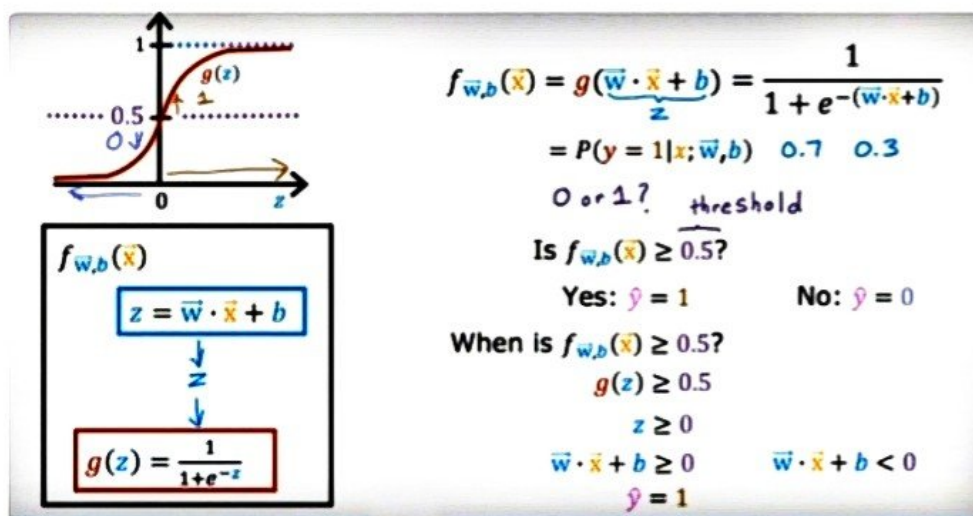
Probability that y is 1,
given input $\bar{x}$, parameters $\bar{w}, b$

$$P(y = 0) + P(y = 1) = 1$$

- ☐ **Interpretazione del Risultato:** L'output $f(x)$ rappresenta la probabilità che $y=1$ dato l'input x . Ad esempio, se $f(x) = 0.7$, significa che c'è il **70% di probabilità** che il tumore sia maligno ($y=1$). In notazione matematica: $P(y=1 | x; w, b)$

Interpretazione Probabilistica

Uno degli aspetti più potenti della regressione logistica è che il suo output può essere interpretato direttamente come una probabilità. Questo rende il modello non solo predittivo, ma anche informativo sulla certezza delle sue previsioni.



Come Funziona

L'output $f(x)$ rappresenta $P(y=1 | x)$, cioè la probabilità che $y=1$ condizionata ai valori delle features x .

Poiché le probabilità devono sommare a 1:

$$P(y = 0) + P(y = 1) = 1$$

Se $f(x) = 0.7$, automaticamente $P(y=0) = 0.3$

Esempio Pratico: Diagnosi Tumore

Se per un paziente il modello calcola $f(x) = 0.7$:

- **70% di probabilità** che il tumore sia maligno ($y=1$)
- **30% di probabilità** che il tumore sia benigno ($y=0$)

Prendere la Decisione

Tipicamente, usiamo una **soglia di decisione** a 0.5:

- Se $f(x) \geq 0.5 \rightarrow$ prediciamo $y=1$ (positivo)
- Se $f(x) < 0.5 \rightarrow$ prediciamo $y=0$ (negativo)

La soglia può essere adattata in base al contesto applicativo.

Esempio Numerico: w e b in Azione

Per capire concretamente come i parametri w e b influenzano le previsioni del modello, analizziamo un esempio numerico con un singolo feature. Consideriamo un modello semplice dove $w = 1$ e $b = -3$, quindi la nostra funzione diventa: $z = x - 3$

Caso 1: $x = 1$

$$z = 1 - 3 = -2$$

$$g(-2) \approx 0.12$$

Previsione: $\hat{y} = 0$

(Tumore Benigno)

Il modello è abbastanza sicuro che sia benigno.

1

Caso 3: $x = 5$

$$z = 5 - 3 = 2$$

$$g(2) \approx 0.88$$

Previsione: $\hat{y} = 1$

(Tumore Maligno)

Il modello è abbastanza sicuro che sia maligno.

3

2

Caso 2: $x = 3$

$$z = 3 - 3 = 0$$

$$g(0) = 0.5$$

Soglia di Incertezza

Il modello è completamente incerto. Siamo esattamente al confine tra le due classi (decision boundary).

Osservazione importante: Il valore di z determina quanto siamo "sicuri" della classificazione. Valori di z molto negativi o molto positivi indicano alta confidenza, mentre valori vicini a zero indicano incertezza.

Laboratorio: Esplorando la Funzione Sigmoidale

Ora che abbiamo compreso la teoria dietro la funzione sigmoide e come essa trasforma i valori lineari in probabilità, è il momento di vederla in azione! In questo laboratorio interattivo, avrai l'opportunità di manipolare direttamente i parametri e osservare come influenzano la curva sigmoide.

Esploreremo visivamente come la funzione sigmoide mappa i valori di input (z) su un intervallo da 0 a 1, e come la forma a "S" della curva sia ideale per la classificazione, permettendo al modello di esprimere la "fiducia" nelle sue previsioni.

 Mettiti comodo e preparati a sperimentare! Capire la sigmoide è fondamentale per padroneggiare la Regressione Logistica.

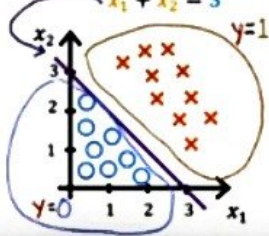
Il Decision Boundary

Il decision boundary (confine decisionale) è la linea, superficie o ipersuperficie che separa le regioni dello spazio delle features in cui il modello predice classi diverse. È un concetto fondamentale per visualizzare e comprendere come il modello prende le sue decisioni.

Decision boundary

$$f_{w,b}(\vec{x}) = g(z) = g(\underbrace{w_1}_{1}x_1 + \underbrace{w_2}_{1}x_2 + \underbrace{b}_{-3})$$

Decision boundary $z = \vec{w} \cdot \vec{x} + b = 0$
 $z = x_1 + x_2 - 3 = 0$
 $x_1 + x_2 = 3$



Definizione Matematica

Il decision boundary si trova dove $f(x) = 0.5$, che corrisponde al punto in cui:

$$z = w \cdot x + b = 0$$

In questo punto, il modello è completamente incerto tra le due classi.

1

Esempio con 2 Features

Consideriamo un modello con due features x_1 e x_2 , e parametri $w_1=1$, $w_2=1$, $b=-3$:

$$z = x_1 + x_2 - 3 = 0$$

2

Equazione della Retta

Risolvendo per il decision boundary otteniamo:

$$x_1 + x_2 = 3$$

Questa è l'equazione di una retta nello spazio 2D.

3

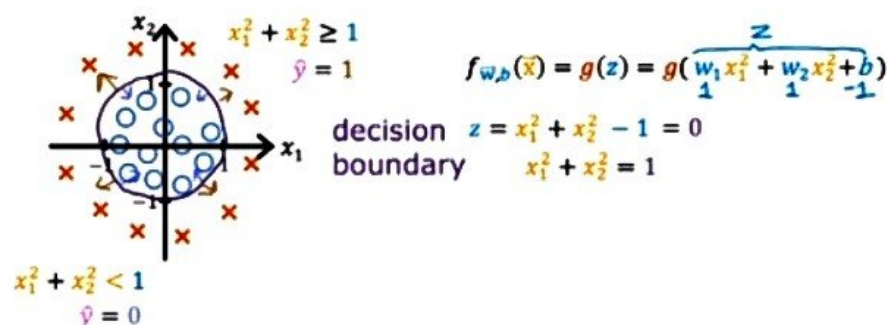
Interpretazione Geometrica

I punti sopra la retta ($x_1 + x_2 > 3$) avranno $\hat{y} = 1$, mentre i punti sotto la retta ($x_1 + x_2 < 3$) avranno $\hat{y} = 0$.

Decision Boundary Non Lineari

La regressione logistica non è limitata a confini decisionali lineari (rette o piani). Quando i dati non sono linearmente separabili, possiamo utilizzare **features polinomiali** per creare decision boundary con forme più complesse che si adattano meglio alla struttura naturale dei dati.

Non-linear decision boundaries



Esempio: Decision Boundary Circolare

Supponiamo di voler separare i punti all'interno di un cerchio da quelli all'esterno. Possiamo usare features polinomiali quadratiche:

$$z = x_1^2 + x_2^2 - 1$$

Il decision boundary si trova dove $z = 0$:

$$x_1^2 + x_2^2 = 1$$

Questa è l'equazione di un **cerchio** con raggio 1 centrato nell'origine!

Come Funziona

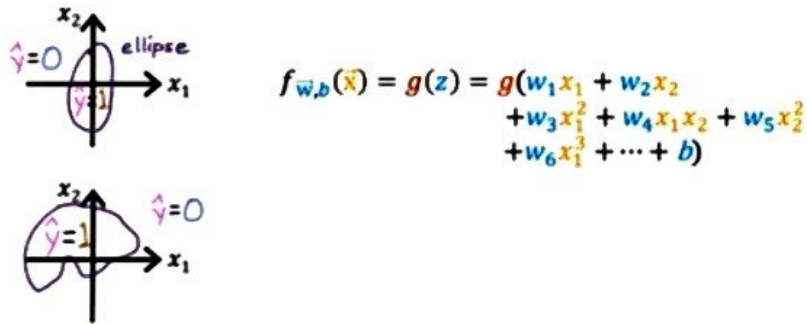
- **Dentro il cerchio:** Per punti dove $x_1^2 + x_2^2 < 1$, avremo $z < 0$, quindi $f(x) < 0.5$ e $\hat{y} = 0$
- **Sul cerchio:** Esattamente $x_1^2 + x_2^2 = 1$, quindi $z = 0$, $f(x) = 0.5$ (incertezza massima)
- **Fuori dal cerchio:** Per punti dove $x_1^2 + x_2^2 \geq 1$, avremo $z \geq 0$, quindi $f(x) \geq 0.5$ e $\hat{y} = 1$

☐ Questo dimostra come la regressione logistica, pur essendo basata su una funzione lineare di z , possa creare confini decisionali non lineari quando usiamo trasformazioni polinomiali delle features originali.

Decision Boundary Complessi

Con features polinomiali di grado superiore, possiamo creare decision boundary estremamente complessi che si adattano a pattern di dati molto articolati. Tuttavia, questa flessibilità ha un costo: il rischio di overfitting.

Non-linear decision boundaries



Polinomi di Alto Grado

Possiamo costruire modelli con termini polinomiali di grado 3, 4 o superiore:

$$f(x) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 + \dots)$$

Forme Possibili

Questo permette di creare decision boundary con forme molto varie:

- Ellissi e parabole
- Forme irregolari e asimmetriche
- Confini con curve multiple
- Regioni di decisione disconnesse

⚠ Il Rischio

Maggiore flessibilità significa anche maggiore rischio di **overfitting**: il modello potrebbe adattarsi troppo bene ai dati di training, inclusi rumore e outliers, perdendo la capacità di generalizzare su nuovi dati.

Principio guida: Il decision boundary dovrebbe essere *abbastanza complesso* da catturare i pattern reali nei dati, ma *abbastanza semplice* da generalizzare bene. Trovare questo equilibrio è una delle sfide chiave del Machine Learning.



Laboratorio: Esplorando i Decision Boundary

Ora che abbiamo esplorato la teoria dei decision boundary lineari e non lineari, è tempo di metterli alla prova in un ambiente interattivo. Questo laboratorio ti permetterà di visualizzare e manipolare direttamente i confini decisionali, offrendoti una comprensione pratica di come il modello logistico separa le classi.



Confini Lineari

Aggiusta i parametri $\mathbf{w}$ e $\mathbf{b}$ per vedere come una semplice retta separa i punti dati.

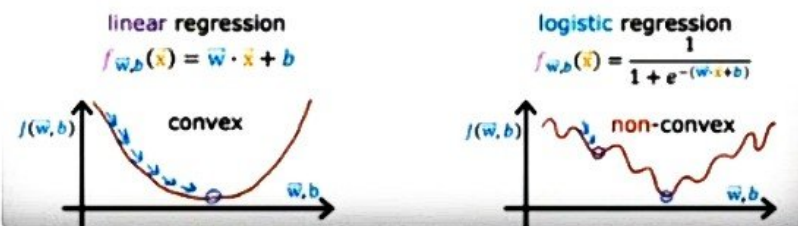
- ☐ L'interazione con i decision boundary è fondamentale per sviluppare un'intuizione solida su come i modelli di classificazione prendono le loro decisioni.

Perché NON Usare l'Errore Quadratico (MSE)

Nella regressione lineare, abbiamo usato l'errore quadratico medio (MSE - Mean Squared Error) come funzione di costo. Tuttavia, per la regressione logistica, l'MSE crea problemi matematici significativi che rendono l'ottimizzazione estremamente difficile o addirittura impossibile.

Squared error cost

$$J(\bar{w}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{w,b}(\bar{x}^{(i)}) - y^{(i)})^2$$



A sinistra: funzione convessa (regressione lineare). A destra: funzione non convessa (logistica con MSE)

Il Problema Fondamentale

Quando combiniamo l'MSE con la funzione sigmoide, otteniamo una funzione di costo **non convessa**, caratterizzata da:

- **Molti minimi locali:** zone in cui la funzione sembra minima, ma non lo è globalmente
- **Plateau e zone piatte:** regioni dove il gradiente è quasi zero
- **Gradient descent instabile:** l'algoritmo può facilmente "bloccarsi" in un minimo locale

Regressione Lineare con MSE

La funzione di costo è **convessa**: ha un solo minimo globale, come una valle o una scodella. Il gradient descent converge sempre al minimo globale.

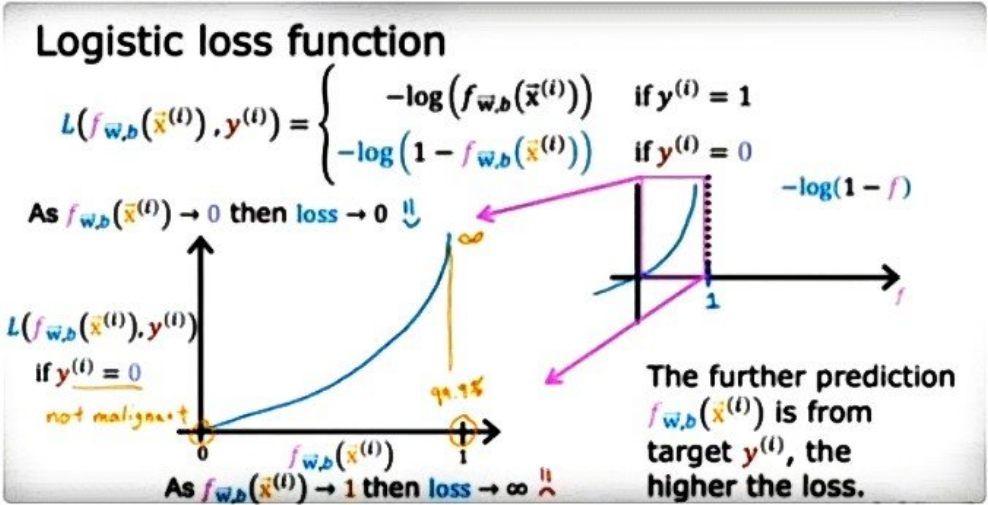
Regressione Logistica con MSE

La funzione diventa **non convessa**: piena di "colline" e "valli" (minimi locali). Il gradient descent può bloccarsi in qualsiasi valle, senza mai trovare il minimo globale vero.

Questo è il motivo per cui abbiamo bisogno di una funzione di costo diversa, progettata specificamente per la classificazione: la **Log Loss** o **Binary Cross-Entropy**.

La Loss Function Logaritmica

La soluzione al problema dell'MSE nella classificazione è una funzione di loss basata sui logaritmi. Questa funzione è progettata specificamente per penalizzare in modo appropriato gli errori di classificazione, mantenendo allo stesso tempo la convessità necessaria per l'ottimizzazione.



Caso 1: $y = 1$ (Classe Positiva)

$$L = -\log(f(x))$$

Comportamento:

- Se $f(x) \rightarrow 1$: la previsione è corretta e sicura. Loss $\rightarrow 0$ (nessuna penalità)
- Se $f(x) \rightarrow 0.5$: la previsione è incerta. Loss moderata
- Se $f(x) \rightarrow 0$: la previsione è sbagliata e sicura. Loss $\rightarrow \infty$ (penalità massima)

Caso 2: $y = 0$ (Classe Negativa)

$$L = -\log(1 - f(x))$$

Comportamento:

- Se $f(x) \rightarrow 0$: la previsione è corretta e sicura. Loss $\rightarrow 0$ (nessuna penalità)
- Se $f(x) \rightarrow 0.5$: la previsione è incerta. Loss moderata
- Se $f(x) \rightarrow 1$: la previsione è sbagliata e sicura. Loss $\rightarrow \infty$ (penalità massima)

Principio chiave: La loss function logaritmica "punisce severamente" le previsioni sbagliate fatte con alta sicurezza. Se il modello è molto sicuro di una previsione ma sbaglia, la penalità è enorme. Se è incerto, la penalità è più moderata.

Cost

$$\begin{aligned} \text{cost} \\ J(\bar{w}, b) &= \frac{1}{m} \sum_{i=1}^m \underbrace{L(f_{\bar{w}, b}(\bar{x}^{(i)}), y^{(i)})}_{\text{loss}} \\ &= \begin{cases} -\log(f_{\bar{w}, b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 1 \text{ } \text{Convex} \\ -\log(1 - f_{\bar{w}, b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases} \quad \begin{matrix} \text{can reach a} \\ \text{global minimum} \end{matrix} \end{aligned}$$

La Cost Function Completa

La funzione di costo $J(w, b)$ per la regressione logistica è semplicemente la media delle loss individuali calcolate su tutti gli esempi del training set. Questa aggregazione mantiene tutte le proprietà desiderabili della loss logaritmica, in particolare la **convessità**.

Definizione Matematica

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}), y^{(i)})$$

Dove:

- m è il numero totale di esempi nel training set
- L è la loss logaritmica per un singolo esempio
- $f(x^{(i)})$ è la previsione del modello per l'esempio i -esimo
- $y^{(i)}$ è l'etichetta vera dell'esempio i -esimo

Proprietà Fondamentale

Convessità Garantita

Grazie alla scelta della loss logaritmica, questa funzione di costo è convessa: ha un solo minimo globale, con una forma a "scodella".

Il Gradient Descent convergerà sempre al minimo globale!

- ☐ La convessità è una proprietà matematica cruciale: significa che non ci sono "trappole" (minimi locali) in cui l'algoritmo di ottimizzazione possa rimanere bloccato. Qualunque sia il punto di partenza, il gradient descent troverà sempre la soluzione ottimale.

La Formula Semplificata

Le due formule separate per la loss (una per $y=1$ e una per $y=0$) possono essere elegantemente combinate in un'unica espressione matematica. Questa formula compatta è quella che viene effettivamente implementata nel codice ed è più facile da derivare per il gradient descent.

Simplified loss function

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\bar{w},b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\bar{w},b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = -\underbrace{y^{(i)}}_0 \log(\underbrace{f_{\bar{w},b}(\bar{x}^{(i)})}_{(1-0)}) - (1 - y^{(i)}) \log(1 - f_{\bar{w},b}(\bar{x}^{(i)}))$$

if $y^{(i)} = 1$:

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = -1 \log(f(\hat{x}))$$

if $y^{(i)} = 0$:

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = - (1-0) \log(1-f(\hat{x}))$$

Formula Unificata della Loss

$$L(f(x), y) = -y \log(f(x)) - (1 - y) \log(1 - f(x))$$

Questa singola formula copre entrambi i casi automaticamente, sfruttando il fatto che y può valere solo 0 o 1.

Come Funziona il Trucco

Quando $y = 1$:

- Il termine $(1-y)$ diventa 0
- Il secondo termine si annulla
- Rimane: $L = -\log(f(x))$ ✓

Quando $y = 0$:

- Il termine y diventa 0
- Il primo termine si annulla
- Rimane: $L = -\log(1-f(x))$ ✓

Questa formulazione è particolarmente elegante perché sfrutta le proprietà algebriche dei valori binari (0 e 1) per "spegnere" automaticamente il termine non necessario, senza dover usare condizioni if-then.

La Cost Function Finale

Sostituendo la formula semplificata della loss nella definizione di costo medio, otteniamo l'espressione finale della funzione di costo per la regressione logistica. Questa è la funzione che dobbiamo minimizzare per addestrare il nostro modello.

Simplified cost function

$$\begin{aligned} \text{loss} \\ L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) &= -y^{(i)} \log(f_{\bar{w},b}(\bar{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\bar{w},b}(\bar{x}^{(i)})) \\ \text{cost} \\ J(\bar{w}, b) &= \frac{1}{m} \sum_{i=1}^m [L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)})] \quad \text{convex (single global minimum)} \\ &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\bar{w},b}(\bar{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\bar{w},b}(\bar{x}^{(i)}))] \end{aligned}$$

Formula Completa

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f(x^{(i)})) + (1 - y^{(i)}) \log(1 - f(x^{(i)}))]$$

Questa è la funzione obiettivo che vogliamo minimizzare rispetto ai parametri w e b .

Nome Tecnico

Questa funzione è conosciuta come:

- **Binary Cross-Entropy Loss**
- **Log Loss**
- **Logistic Loss**

È la funzione di costo standard e universalmente accettata per problemi di classificazione binaria.

Origine Teorica

Questa formula non è arbitraria: deriva dal principio statistico del **Maximum Likelihood Estimation (MLE)**. Minimizzare questa funzione equivale a massimizzare la probabilità che il modello generi correttamente le etichette osservate nei dati di training.

Il Gradient Descent

Ora che abbiamo definito la funzione di costo $J(w,b)$, dobbiamo trovare i valori ottimali dei parametri w e b che la minimizzano. Per fare questo, utilizziamo lo stesso algoritmo iterativo che abbiamo già visto nella regressione lineare: il **gradient descent**.

Gradient descent for logistic regression

```
repeat {  
    looks like linear regression!  
     $w_j = w_j - \alpha \left[ \frac{1}{m} \sum_{i=1}^m (f_{w,b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right]$   
     $b = b - \alpha \left[ \frac{1}{m} \sum_{i=1}^m (f_{w,b}(\vec{x}^{(i)}) - y^{(i)}) \right]$   
} simultaneous updates
```

Same concepts:

- Monitor gradient descent (learning curve)
- Vectorized implementation
- Feature scaling

Linear regression $f_{w,b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

Logistic regression $f_{w,b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$

L'Algoritmo

Il gradient descent aggiorna iterativamente i parametri seguendo la direzione opposta al gradiente:

```
Repeat {  
     $w_j = w_j - \alpha * \partial/\partial w_j J(w,b)$   
     $b = b - \alpha * \partial/\partial b J(w,b)$   
}
```

Dove:

- α (alpha) è il learning rate
- $\partial/\partial w_j$ è la derivata parziale rispetto a w_j

❏ **Importante: Aggiornamento Simultaneo!** Tutti i parametri (tutti i w_j e b) devono essere aggiornati contemporaneamente nella stessa iterazione. Non dobbiamo usare i nuovi valori di alcuni parametri per calcolare gli aggiornamenti di altri parametri nella stessa iterazione.

Le formule specifiche per le derivate parziali della regressione logistica, quando espresse completamente, hanno una forma molto simile a quelle della regressione lineare, anche se derivano da funzioni di costo diverse.

Confronto: Lineare vs Logistica

A prima vista, le regole di aggiornamento del gradient descent per la regressione lineare e la regressione logistica sembrano identiche. Tuttavia, questa somiglianza superficiale nasconde una differenza fondamentale nella definizione della funzione $f(x)$.

La Formula di Aggiornamento (Identica per entrambe)

$$w_j = w_j - \alpha \cdot \frac{1}{m} \cdot \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Questa è la regola per aggiornare ogni peso w_j . La formula è matematicamente identica sia nella regressione lineare che nella regressione logistica.

MA: $f(x)$ è Definita Diversamente!

Regressione Lineare:

$$f(x) = w \cdot x + b$$

Una semplice combinazione lineare delle features.

Regressione Logistica:

$$f(x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

La combinazione lineare passa attraverso la funzione sigmoide.

Stessa formula, definizione diversa: Questo significa che, anche se le regole di aggiornamento sembrano uguali, il gradient descent sta in realtà ottimizzando funzioni di costo completamente diverse (MSE per la lineare, Binary Cross-Entropy per la logistica) e producendo modelli con comportamenti molto diversi.

A prima vista, le regole di aggiornamento del gradient descent per la regressione lineare e la regressione logistica sembrano identiche. Tuttavia, questa somiglianza superficiale nasconde una differenza fondamentale nella definizione della funzione $f(x)$.

La Formula di Aggiornamento (Identica per entrambe)

$$w_j = w_j - \alpha \cdot \frac{1}{m} \cdot \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Questa è la regola per aggiornare ogni peso w_j . La formula è matematicamente identica sia nella regressione lineare che nella regressione logistica.

...

MA: $f(x)$ è Definita Diversamente!

I

+

Regressione Lineare:

$$f(x) = w \cdot x + b$$

Una semplice combinazione lineare delle features.

Regressione Logistica:

$$f(x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

La combinazione lineare passa attraverso la funzione sigmoide.

I

Il Problema dell'Overfitting (Regressione)

L'overfitting è uno dei problemi più comuni e insidiosi nel Machine Learning. Si verifica quando un modello impara troppo bene i dati di training, inclusi rumore e anomalie, perdendo la capacità di generalizzare su nuovi dati. Vediamo come si manifesta nei problemi di regressione.